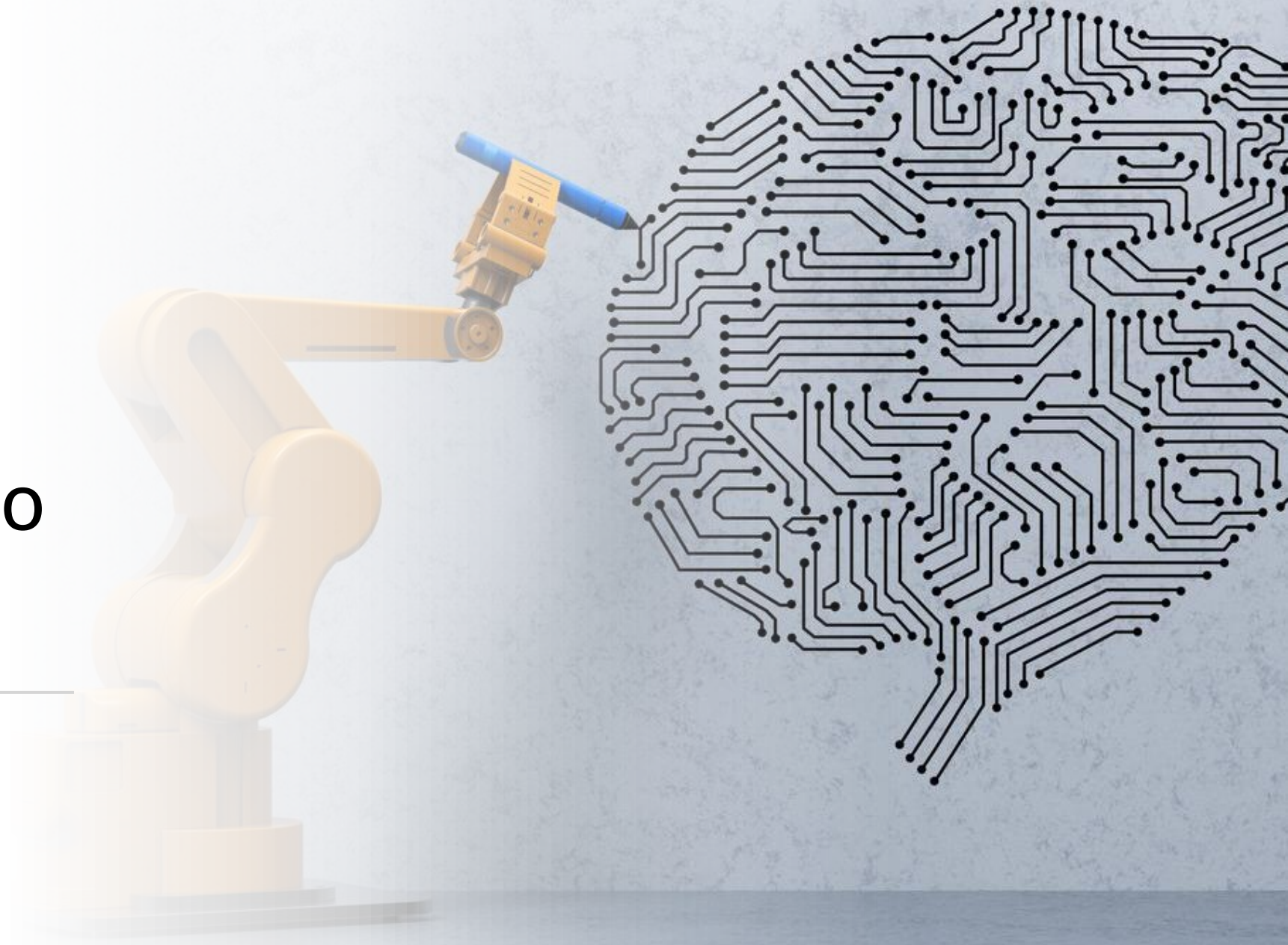
Reconocimiento de Patrones

Clase 12

Regresión Logística

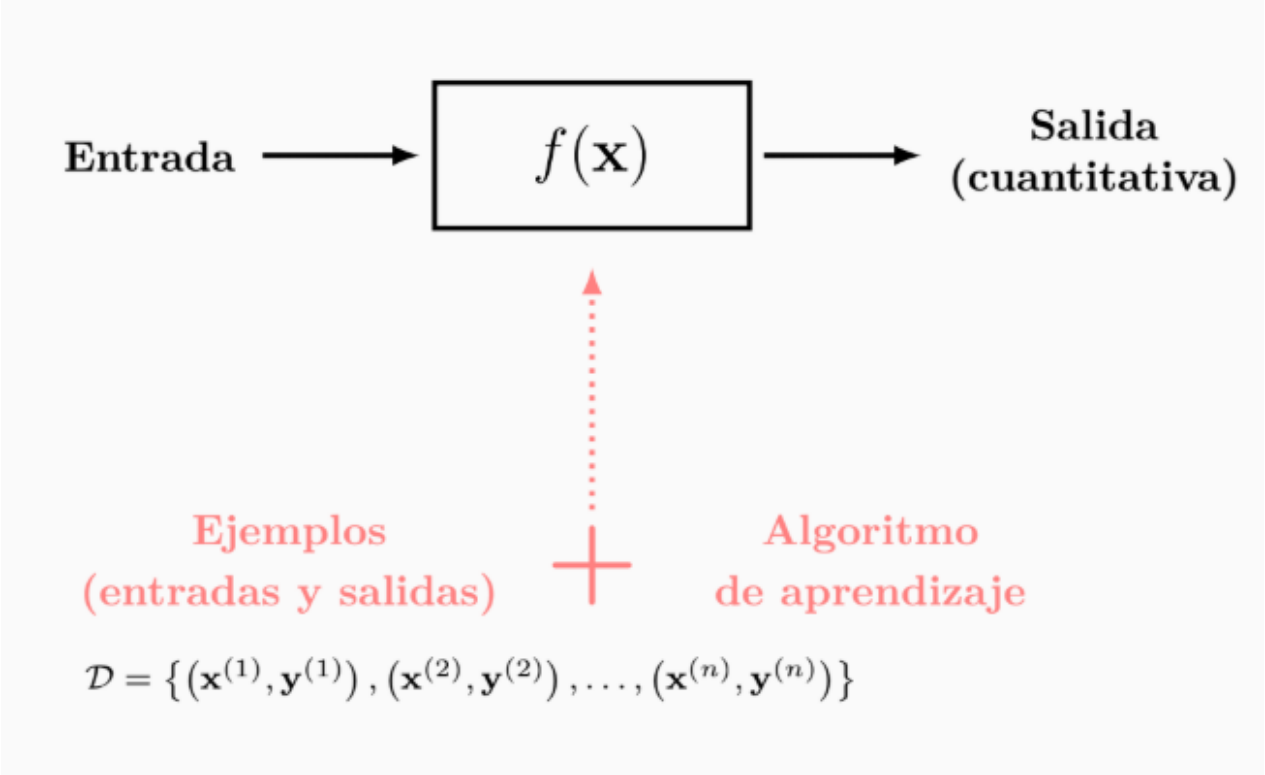


Antes de empezar

- ¿Dudas de la clase anterior?



Regresión

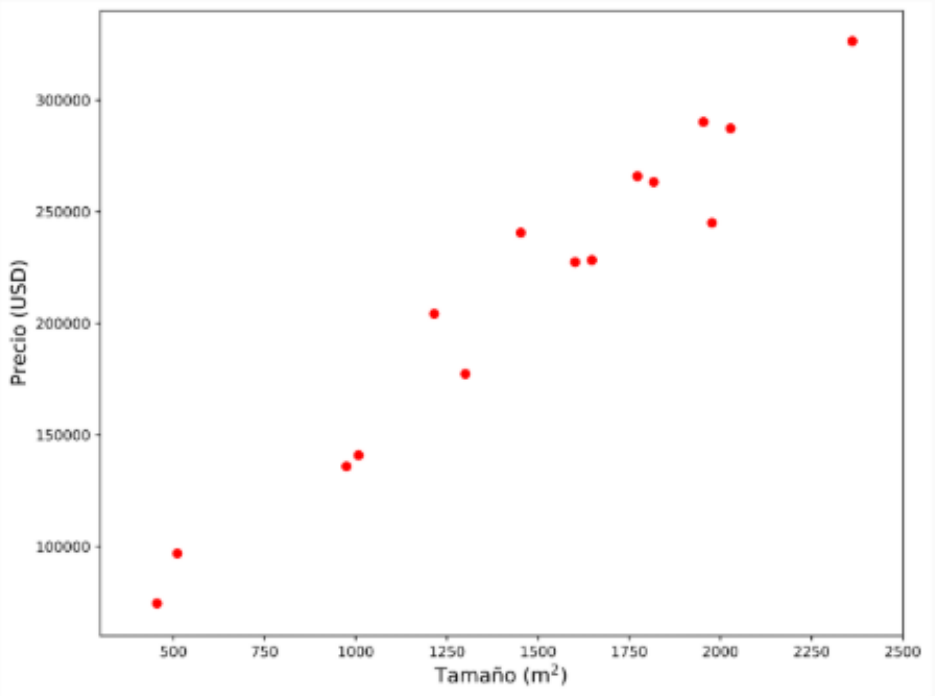


- Salida continua: cuantitativa
- Por ejemplo: predecir temperaturas, precios de autos o casas, etc.

Precios de casas

- ¿Cómo puedo ajustar una función f para modelar la relación entre el tamaño de una casa y el precio de esta?

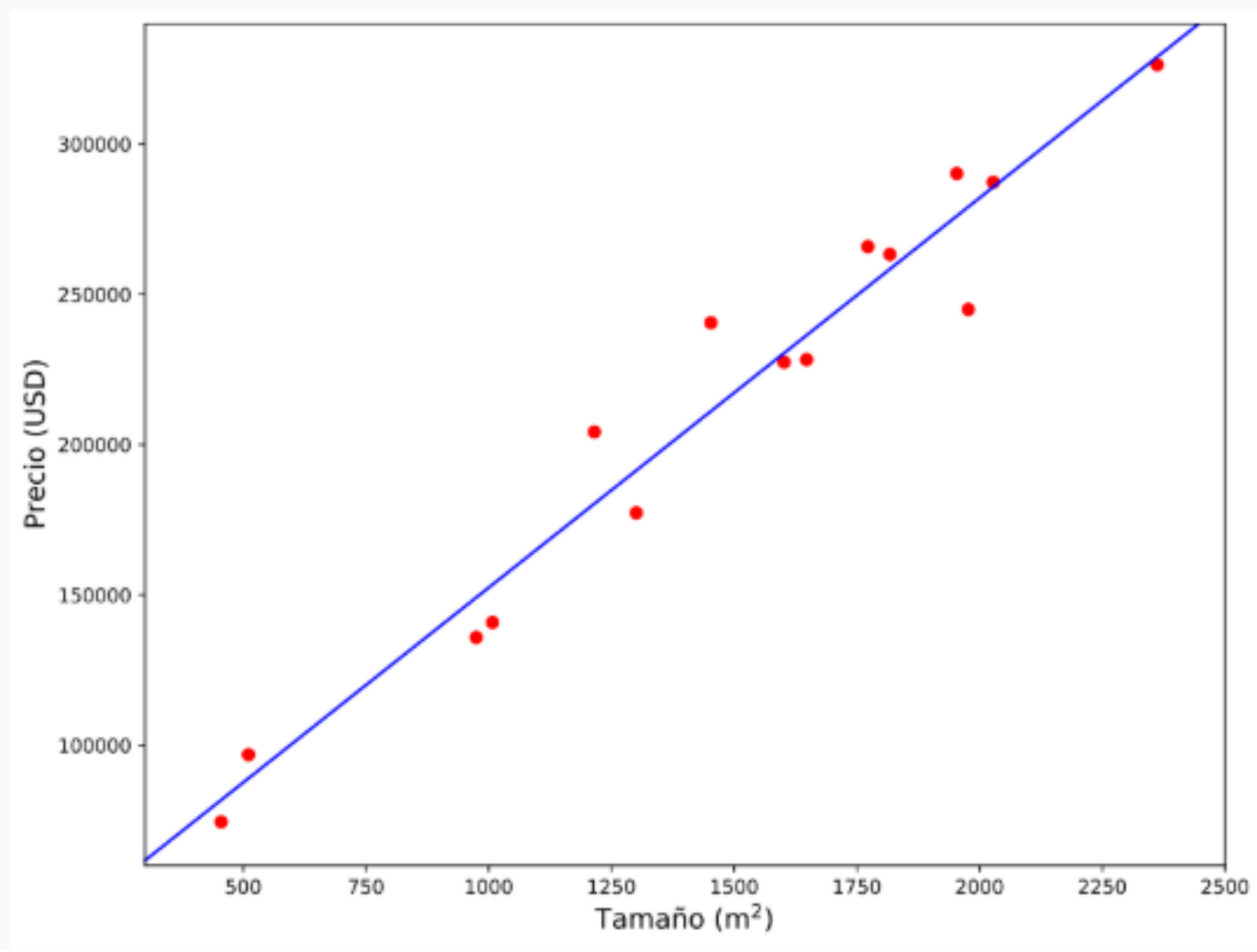
Tamaño (m^2)	Precio (USD)
489.59	489.59
556.08	556.08
570.35	570.35
772.84	772.84
970.95	970.95
1162.00	1162.00
1263.10	1263.10
⋮	⋮



- Se puede suponer una relación lineal entre las variables

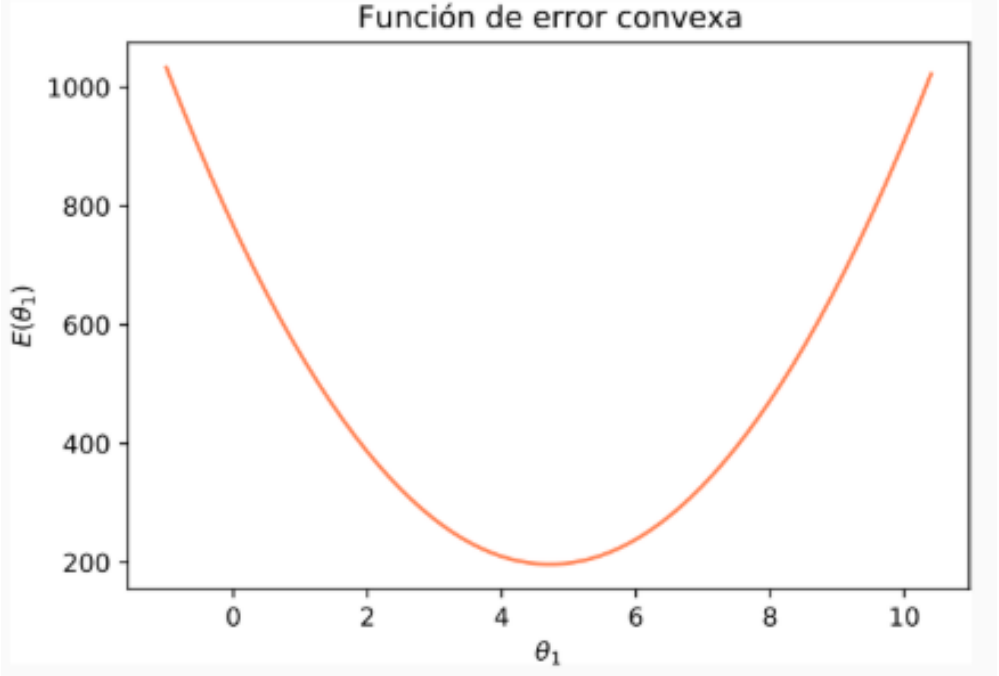
$$f_\theta(x) = \theta_0 + \theta_1 \cdot x$$

- ¿Pero como sabemos que tan bien se ajusta la función a nuestros datos?



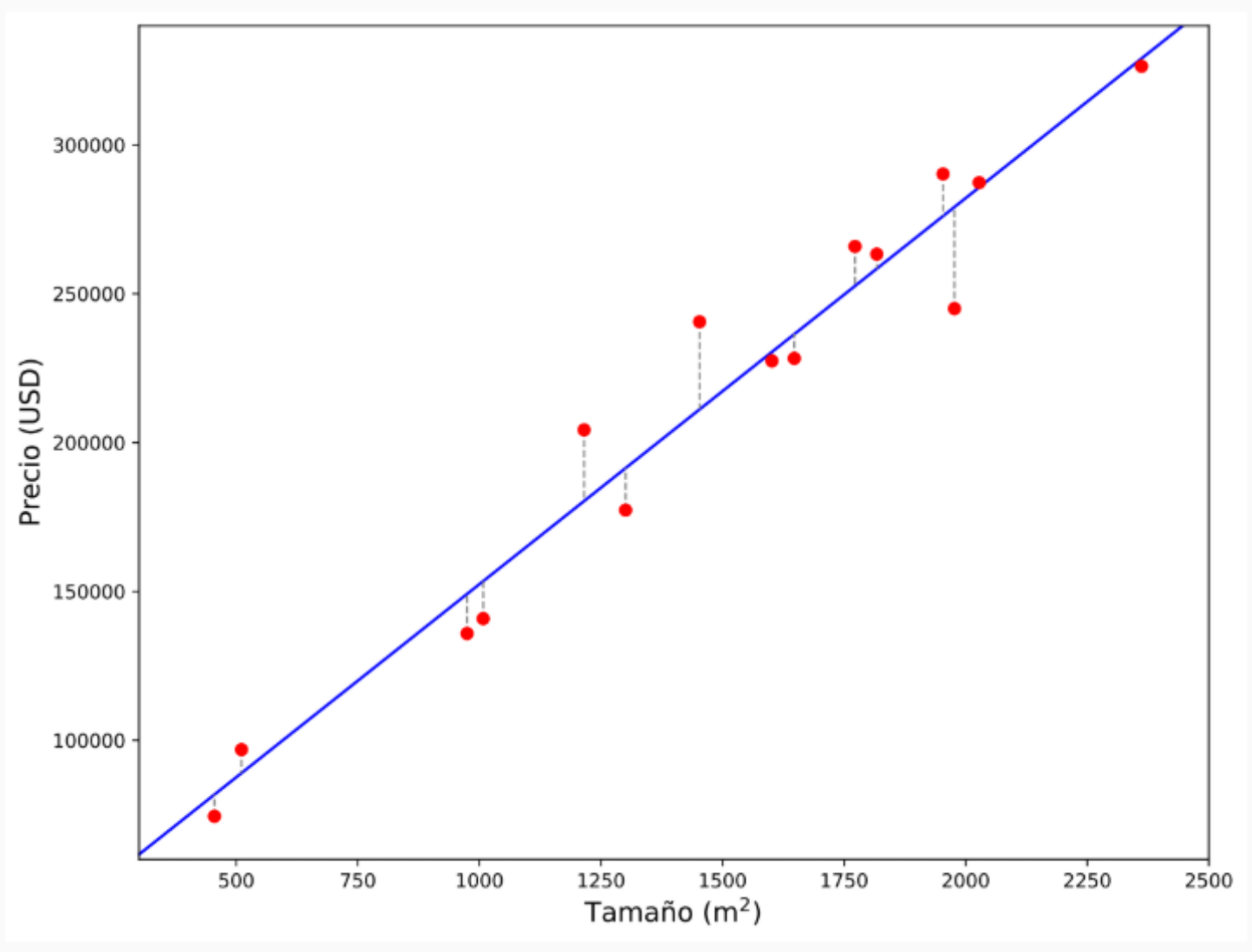
- Se define lo que se conoce como **función de costo**, la cual nos dice que tanto error tenemos cuando comparamos lo predicho con lo real. Una función de costo puede ser la suma de errores cuadrados.

$$\mathrm{E}(\theta) = \frac{1}{2\mathrm{n}} \cdot \sum_{\mathrm{i}=1}^{\mathrm{n}} (\mathrm{f}_{\theta}(\mathrm{x}_{\mathrm{i}}) - \mathrm{y}_{\mathrm{i}})^2$$



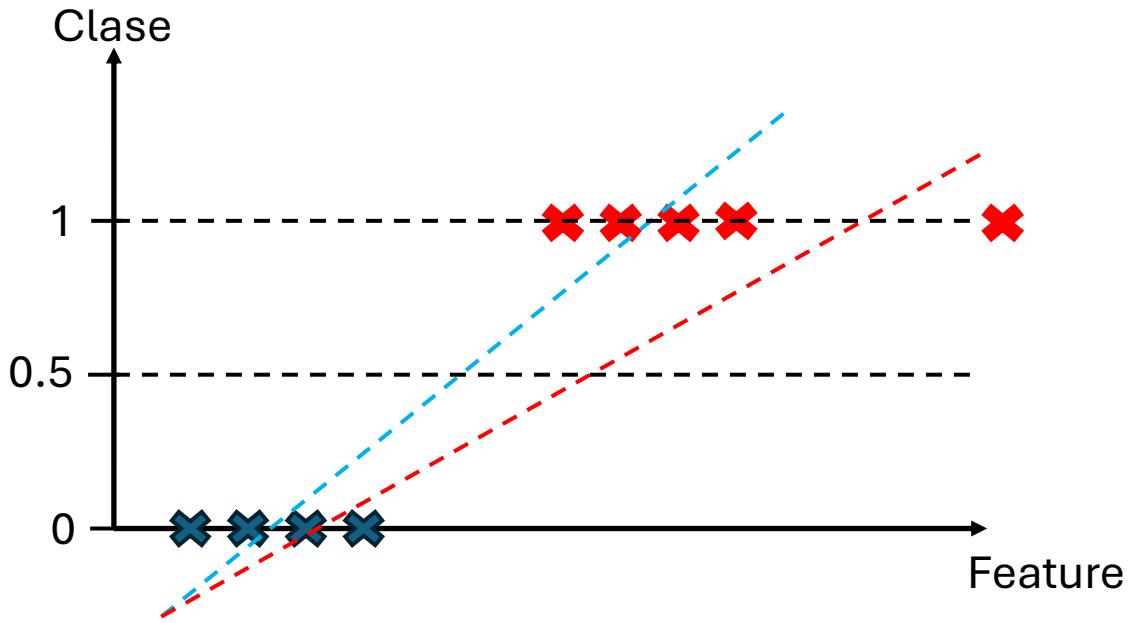
Objetivo: Encontrar el valor de θ que minimice la función de costo $\mathrm{E}(\theta)$.

$$\hat{\theta} = argmin_{\theta} E(\theta)$$



¿Esto nos sirve para clasificación?

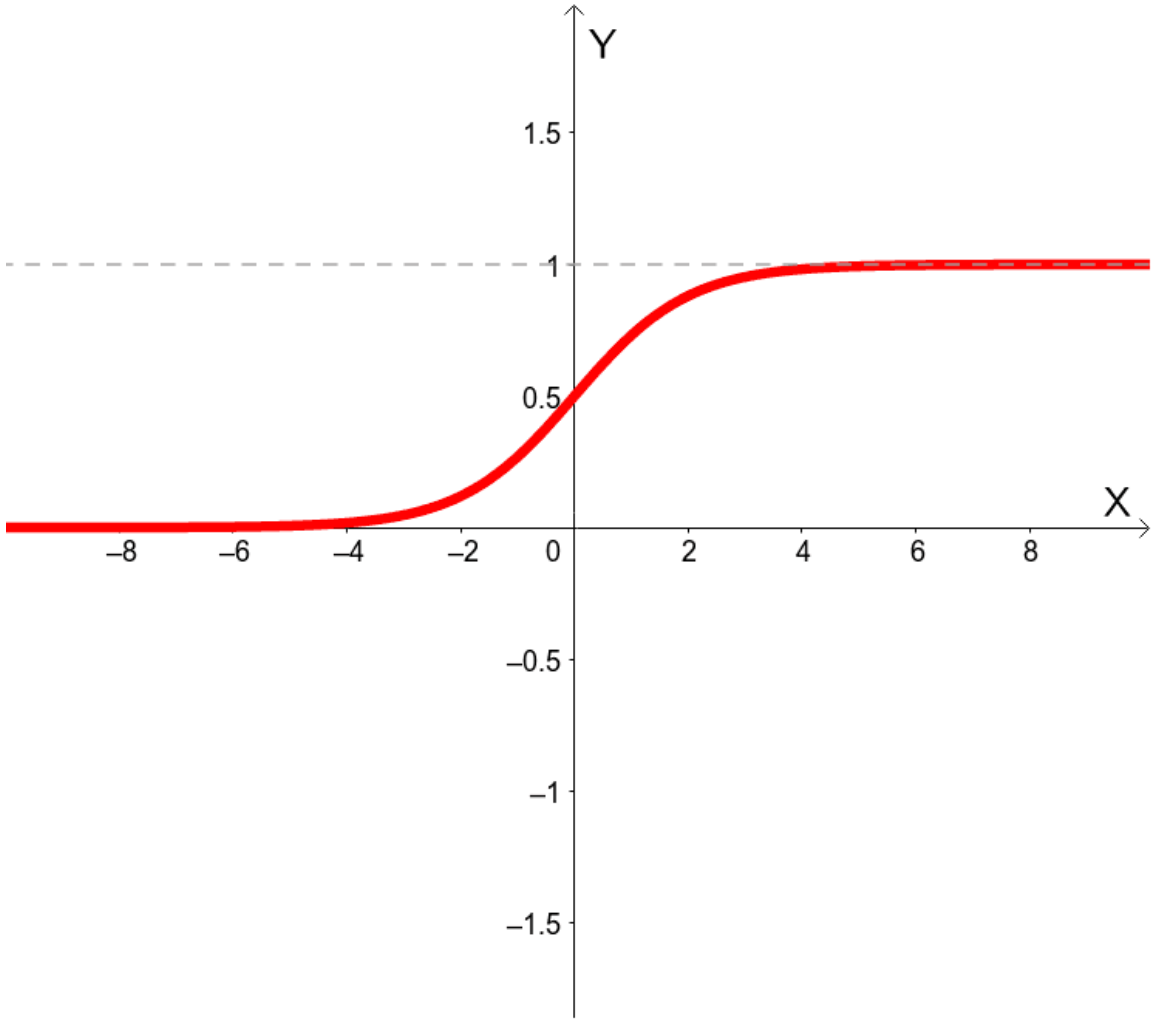
- ¿Se puede modelar la clasificación binaria como un problema de regresión?



Regresión Logística

- Va tener como objetivo estimar la probabilidad de que una observación pertenezca a una categoría. La función que nos permite modelar este comportamiento es la función sigmoide (logística).

$$g(z) = \frac{1}{1 + e^{-z}}$$



Entonces se puede establecer:

$$h_\theta(x) = g(\theta^T \boldsymbol{x}) = \frac{1}{1 + e^{-\theta^T \boldsymbol{x}}} = P(y = 1|x)$$

Así mismo:

$$P(y = 0|x) = 1 - P(y = 1|x)$$

Odds: $\frac{P(y=1|x)}{P(y=0|x)} = e^{\theta^T \boldsymbol{x}} = e^{\theta_0 + \theta_1 x_1 + \cdots + \theta_k x_k}$

Log-odds: $Log\left(\frac{P(y=1|x)}{P(y=0|x)}\right) = \theta^T \boldsymbol{x} = \theta_0 + \theta_1 x_1 + \cdots + \theta_k x_k$

Entonces lo que tenemos es:

- Conjunto de entrenamiento: $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$
- m ejemplos $x \in \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$ $x_0 = 1, y \in \{0,1\}$
- $h_\theta(x) = \frac{1}{1+e^{-\theta^T x}}$

- Ya tenemos la función para modelar la relación y el tipo de datos. Solo nos falta definir la función de costo.



Función de costo

- Podríamos utilizar la suma de errores cuadrados:

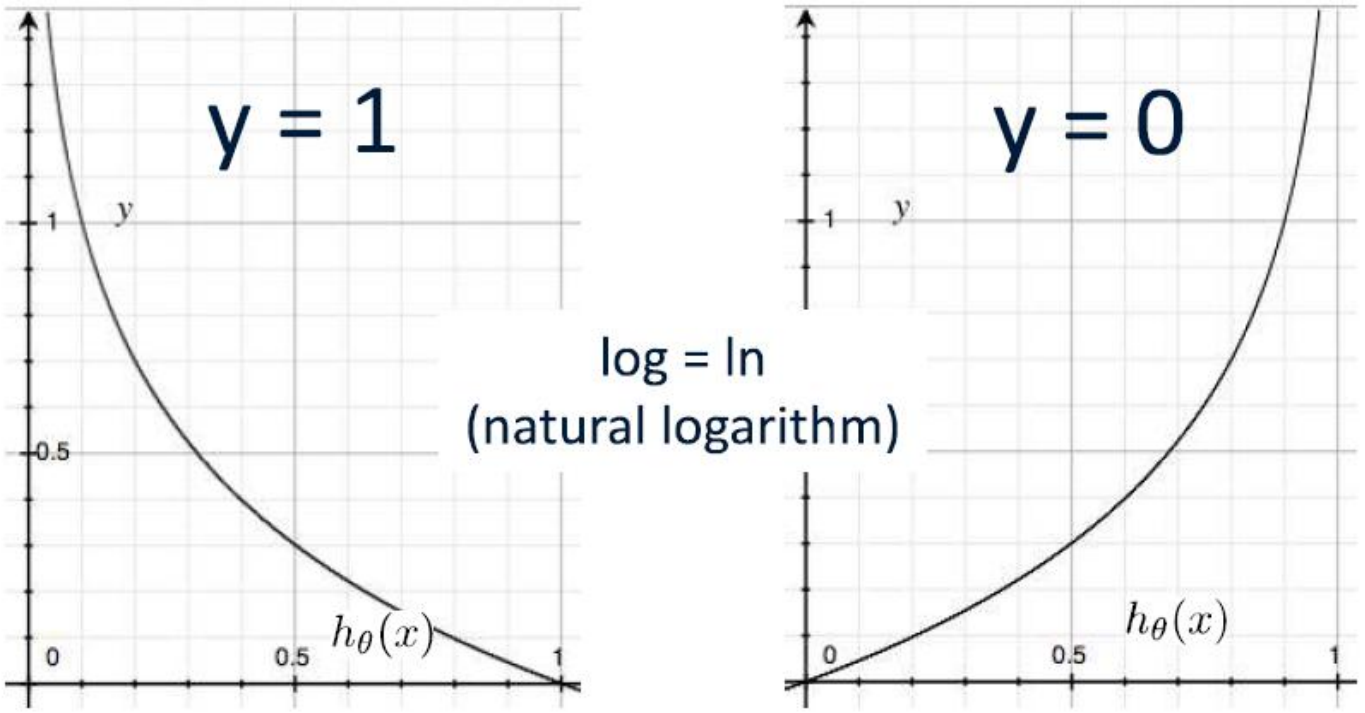
$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} Cost(h_\theta, y)$$

$$Cost(h_\theta, y) = \frac{1}{2}(h_\theta(x) - y)^2$$

El problema es que esta función de costo **no es convexa**. Por lo tanto no es la más adecuada.

Para solucionar este problema se utiliza una función de costo diferente, en este caso se aplica logaritmo natural a la h(x).

$$Cost(h_\theta, y) = \begin{cases} -\log\left(h_\theta(x)\right) & si\ y = 1 \\ -\log\left(1 - h_\theta(x)\right) & si\ y = 0 \end{cases}$$



Entonces la función de costo se puede reescribir como:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m} Cost\left(h_\theta(x^{(i)}), y^{(i)}\right)$$

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_\theta(x^{(i)})\right) + (1 - y^{(i)}) \log\left(1 - h_\theta(x^{(i)})\right)\right]$$

Esta función se conoce como **Entropía Cruzada Binaria.**

Observación	Clase real (y)	p̂ predicha	Pérdida individual
1	1	0.90	$-\log(0.90) = 0.105$
2	1	0.40	$-\log(0.40) = 0.916$
3	0	0.15	$-\log(1-0.15) = 0.163$
4	0	0.80	$-\log(1-0.80) = 1.609$

- Ahora, ¿cómo hacemos para encontrar los parámetros θ?

Se busca minimizar la función de costo:

$$min_{\theta} J(\theta)$$

Para entrenar el modelo se usa el **descenso del gradiente**.

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$\frac{\partial}{\partial \theta_j} J(\theta) = \left[\frac{\partial J}{\partial \theta_0}, \cdots, \frac{\partial J}{\partial \theta_d} \right]$$

¿Cómo se hace este entrenamiento?

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta)$$

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_\theta(x^{(i)})\right) + (1 - y^{(i)}) \log\left(1 - h_\theta(x^{(i)})\right)\right]$$

Derivamos primero la función sigmoide:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\frac{\partial}{\partial z} \frac{1}{1 + e^{-z}} = -(1 + e^{-z})^{-2} e^{-z}(-1) = \frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1 + e^{-z} - 1}{(1 + e^{-z})^2} = \frac{1 + e^{-z}}{(1 + e^{-z})^2} - \frac{1}{(1 + e^{-z})^2}$$

$$= \frac{1}{(1 + e^{-z})} - \frac{1}{(1 + e^{-z})^2} = \frac{1}{(1 + e^{-z})}\left(1 - \frac{1}{(1 + e^{-z})}\right) = \sigma(z)[1 - \sigma(z)]$$

$$\frac{\partial}{\partial \theta_j}\sigma(\theta^T x) = \sigma(\theta^T x)[1-\sigma(\theta^T x)]x \qquad J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_\theta(x^{(i)})\right) + (1-y^{(i)}) \log\left(1-h_\theta(x^{(i)})\right)\right]$$

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} y \log\left(\sigma(\theta^T x)\right) + \frac{\partial}{\partial \theta_j}(1-y)\log\left(1-\sigma(\theta^T x)\right) = \left[\frac{y}{\sigma(\theta^T x)} - \frac{1-y}{1-\sigma(\theta^T x)}\right]\frac{\partial}{\partial \theta_j}\sigma(\theta^T x)$$

$$= \left[\frac{y}{\sigma(\theta^T x)} - \frac{1-y}{1-\sigma(\theta^T x)}\right]\sigma(\theta^T x)[1-\sigma(\theta^T x)]x = \left[\frac{y-\sigma(\theta^T x)}{\sigma(\theta^T x)[1-\sigma(\theta^T x)]}\right]\sigma(\theta^T x)[1-\sigma(\theta^T x)]x$$

$$= [y-\sigma(\theta^T x)]x$$

Repetir hasta la convergencia {

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_j) \qquad para\ j = 0,1,\dots,n$$

}

Repeat {

$$\theta_j := \theta_j - \alpha \left[\frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}\right]$$

(simultaneously update all θ_j)

}

De esta forma la actualización de los parámetros se hace únicamente con la diferencia entre las predicciones y etiquetas reales multiplicada por el vector de características x.

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^{m} \left(h_\theta\left(x^{(i)}\right) - y^{(i)} \right) x_j^{(i)}$$

Observe que para que la actualización se haga de forma correcta, el vector de características **x** debe tener un 1 al inicio para que pueda multiplicarse por el parámetro θ_0

Regularización

Un problema frecuente es que los pesos crecen demasiado y el modelo memoriza el conjunto de entrenamiento en lugar de generalizar.

La solución es la regularización, que añade un término de penalización a la actualización para frenar ese crecimiento:

- L2(Ridge):$\theta_j \coloneqq \theta_j - \alpha \left(\frac{\partial}{\partial \theta_j} J(\theta) \ + \lambda \|\omega\|^2 \right)$
- L1(Lasso):$\theta_j \coloneqq \theta_j - \alpha \left(\frac{\partial}{\partial \theta_j} J(\theta) \ + \lambda \|\omega\|^1 \right)$

Para L2:

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_\theta\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - h_\theta\left(x^{(i)}\right)\right)\right] + \frac{\lambda}{2m}\sum_{j=1}^{n} \theta_j^2$$

$$\theta_j := \theta_j - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right) x_j^{(i)} + \frac{\lambda}{m}\theta_j\right]$$

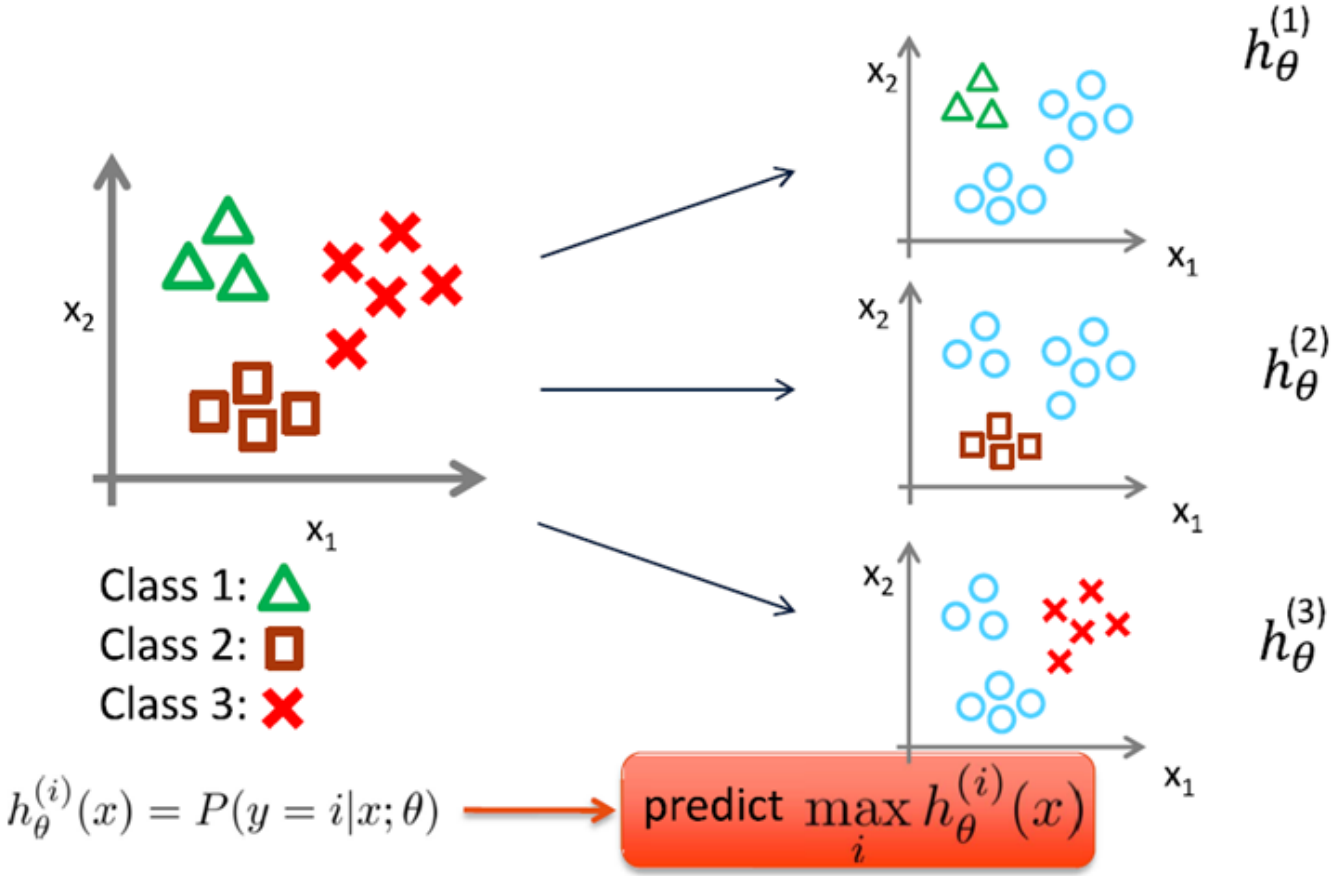
Para L1:

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} y^{(i)} \log\left(h_\theta\left(x^{(i)}\right)\right) + \left(1 - y^{(i)}\right) \log\left(1 - h_\theta\left(x^{(i)}\right)\right)\right] + \frac{\lambda}{m}\sum_{j=1}^{n} \left|\theta_j\right|$$

$$\theta_j := \theta_j - \alpha\left[\frac{1}{m}\sum_{i=1}^{m}\left(h_\theta\left(x^{(i)}\right) - y^{(i)}\right) x_j^{(i)} + \frac{\lambda}{m} sign(\theta_j)\right]$$

Para múltiples clases

- Se entrenan k clasificadores binarios del tipo uno vs el resto



Regresión logística multinomial (Softmax)

- En lugar de K modelos separados, se entrena **un solo modelo** que produce una distribución de probabilidad sobre todas las clases

- Para cada clase k: $z_k = w_k^T x + b_k$
- Luego aplicamos softmax:

$$P(y = C_k \mid x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

$$P(y = C_k \mid x) = \frac{e^{w_k^T x + b_k}}{\sum_{j=1}^{K} e^{w_j^T x + b_j}}$$

- Todas las probabilidades suman 1:

$$\sum_{k} P\left(y = C_k \mid x\right) = 1$$

- La función de costo ahora es **entropía cruzada multiclase**:

$$J = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log(P(y = C_k \mid x_i))$$

$$\hat{y} = argmax_k P(y = C_k \mid x)$$

La softmax es la base para la capa de salida de las **redes neuronales**.

Pesado por clase

En lugar de tratar todos los errores igual, el modelo penaliza **más** equivocarse en la clase minoritaria. Esto se logra modificando la función de costo.

$$J(\theta) = -\frac{1}{m}\left[\sum_{i=1}^{m} w^{(i)}[y^{(i)} \log\left(h_\theta(x^{(i)})\right) + (1 - y^{(i)}) \log\left(1 - h_\theta(x^{(i)})\right)]\right]$$

Donde $w^{(i)}$ es el peso asignado a la clase de ese ejemplo. Un error en un ejemplo de clase minoritaria pesa más → el gradiente lo corrige más agresivamente.

¿Cómo se calculan los pesos?

Los pesos se pueden calcular de forma automática mediante:

$$w_c = \frac{m}{k \cdot m_c}$$

Donde m es el total de ejemplos, k es el número de clases y m_c es el número de ejemplos de la clase c.

Por ejemplo: $n_0 = 950$ y $n_1 = 50$

w_0 = 1000 / (2 × 950) = **0.526** | w_1 = 1000 / (2 × 50) = **10.000** | un error en clase 1 pesa **19.0×** más que uno en clase 0

En python

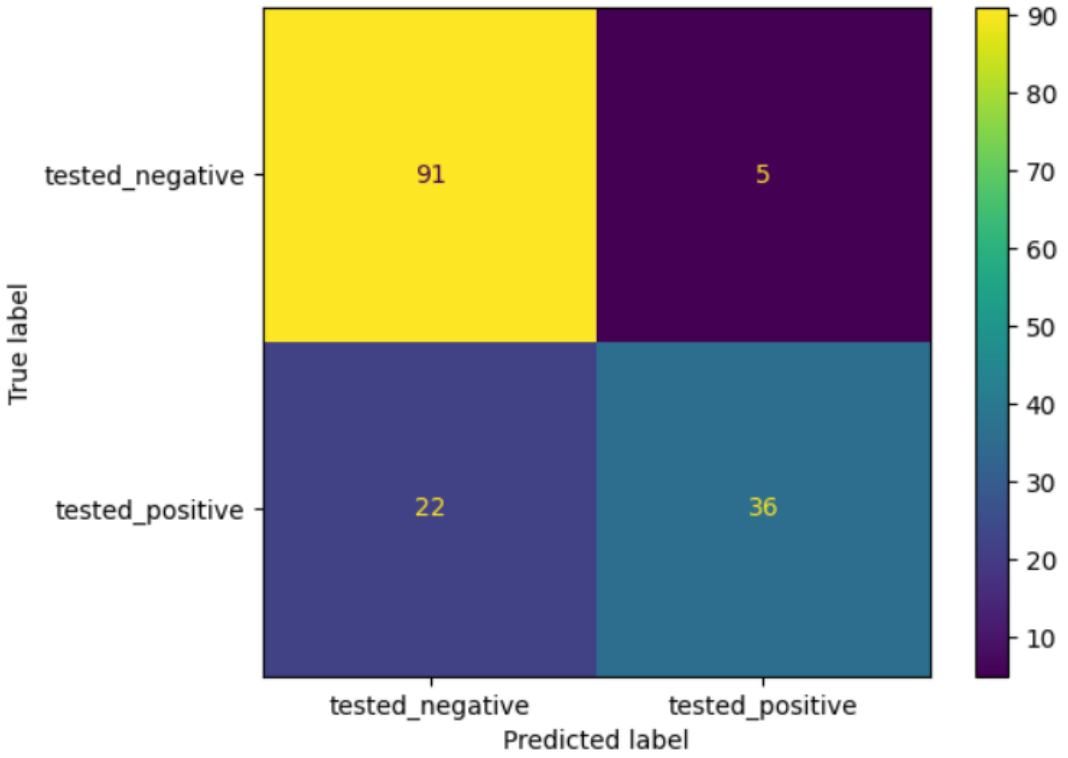
```python
def normalize(data):
    normalized_data = (data-data.min())/(data.max()-data.min())
    return normalized_data

def hipotesis(X, theta):
    inter = X @ theta
    h = 1 / (1 + np.exp(-inter))
    return h

def descensoGradiente(x, y, theta, alpha, m, maxsteps):
    xTrans = x.transpose()
    for i in range(maxsteps):
        #calculo de las y predichas
        y_pred = hipotesis(x, theta)

        #actualizacion de los parametros
        gradiente = (1/m) * (xTrans @ (y_pred - y))
        theta = theta - alpha * gradiente

    return theta
```





Consideraciones finales

- Puede utilizarse en predicción de propensión a compra, detención de fraude y predicción de enfermedades.
- Entrena muy rápido, tiene una salida interpretable, su función de costo es convexa, requiere de pocos hiperparámetros y funciona bien con pocos datos.
- Solo aprende fronteras de decisión lineales, sensible a outliers extremos y características irrelevantes, la multicolinealidad entre características desestabiliza los coeficientes.
- Normalizar o estandarizar los datos antes de entrenar, usar siempre alguna regularización, si hay desbalance de clase conviene hacer un pesado por clase.

Para la próxima clase...

SVM

